

HTML Ki Poori Guide

Zero se leke advanced tags tak — har topic Roman Urdu mein, examples ke saath. Ye notes video course ka mukammal khulasa hain.

CONTENTS

1. HTML Kya Hai — Introduction
2. Text Editor Choose Karna
3. Pehli HTML File Banana (Skeleton)
4. Basic HTML Tags (Meta, Headings, Paragraph, Bold/Italic)
5. Comments
6. Styles aur Colors
7. Layout Tags (Header, Main, Footer, Nav, Section, Article, Aside)
8. Links (Anchor Tag)
9. Images
10. Videos (Local + YouTube Embed)
11. Lists (Ordered, Unordered, Description)
12. Tables
13. Containers — Div aur Span (Block vs Inline)
14. Input Tags aur Forms
15. iFrames
16. Meta Tags (SEO)

1 HTML Kya Hai — Introduction

HTML ka matlab hai **HyperText Markup Language**. Ye ek language hai jo web pages banane ke liye use hoti hai. Internet par jitni bhi websites hain, unme se almost har page HTML mein likha gaya hai.

HTML mein hum **tags** define karte hain, jo website ka layout, look aur feel decide karte hain. Agar आपको pata ho ke tags kya hote hain aur unko kahan aur kaise use karna hai, to aap apni khud ki website bana sakte hain.

HTML seekhna utna mushkil nahi jitna log samajhte hain. Resume par HTML likhna आपको job dilwane mein bhi help kar sakta hai.

2 Text Editor Choose Karna

HTML likhne ke liye आपको ek **text editor** chahiye — koi aisi app jo text files ko edit aur save kar sake. HTML files ka extension `.html` hota hai (jaise `.txt` ki tarah, bas alag extension).

Word processors (Microsoft Word, Google Docs, LibreOffice) HTML likhne ke liye use **na karein** — ye overkill hain aur HTML ke liye nahi bane.

Beginners ke liye (Simple)

- Windows → **Notepad**
- Mac → **TextEdit**
- Simple editor आपको fundamentals par focus karne mein madad karta hai — har tag khud type karna padta hai.

Advanced Editors

- Atom, Sublime Text, Brackets, Notepad++, WebStorm
- Syntax highlighting, themes, add-ons deti hain
- Thori complex hoti hain — beginners baad mein switch karein

Recommendation: Pehle simple text editor se shuru karein, jab HTML ka grasp ho jaye tab advanced editor pe move karein. Best editor wahi hai jo आपको sabse acha lage — logon ki bahaso mein na parhein.

3 Pehli HTML File Banana

Sabse pehle ek project folder banayein, uske andar ek file banayein aur naam rakhein `index.html` . **index.html** ek special naam hai — jab ye file kisi folder ke root mein ho, to browser ise us website ka home page samajhta hai.

Basic Skeleton

```
<!DOCTYPE html>
<html>
  <head>
    <title>Mike's Website</title>
  </head>
  <body>
    Hello World
  </body>
</html>
```

Tag	Kaam
<code><!DOCTYPE html></code>	Batata hai ke ye document HTML type ka hai (single tag, koi closing nahi)
<code><html>...</html></code>	Sab se bara (highest-level) container tag — pura website isi ke andar hota hai
<code><head>...</head></code>	Document ke baare mein metadata — title, description, imports, etc. (jaise "settings")
<code><body>...</body></code>	Actual content jo browser render karta hai — sab kuch jo user dekhta hai
<code><title>...</title></code>	Head ke andar, browser tab par jo naam show hota hai

Container Tags kya hote hain?

Container tags mein **opening tag** aur **closing tag** dono hote hain (closing tag mein forward slash `/` lagta hai). In ke andar aap dusre tags ya content daal sakte hain.

Parent, Child, Sibling — Family Terminology

- `<html>` is **parent** of `<head>` aur `<body>`
- `<head>` aur `<body>` ek dusre ke **siblings** hain (same parent share karte hain)
- Agar `<body>` ke andar `<p>` daalein, to wo `<html>` ka **grandchild** hoga

Indentation important hai: jo tag kisi dusre tag ke andar ho, use tab/spaces se indent karein — is se relationships samajhna asaan hota hai.

File ko Browser mein Kholna

File pe right-click → Open With → koi bhi web browser (Chrome, etc). Browser HTML tags ko parse karke unhe visually render karta hai.

4 Basic HTML Tags

Meta Tags (Head mein)

```
<meta charset="UTF-8">
<meta name="description" content="This is an awesome website">
```

Attributes / Properties: Tags ko extra information dene ke liye hum *attributes* pass karte hain (jaise `charset`, `name`, `content`). Ye tag ke andar likhe jate hain aur us tag ka kaam poora karne mein help karte hain.

Headings — h1 se h6

```
<h1>Mike's Website</h1> // sab se bara
<h2>...</h2>
<h6>...</h6> // sab se chota
```

Best practice: headings hierarchy mein use karein — pehle sirf ek `h1` (main title), phir `h2` sections ke liye, phir `h3` sub-sections ke liye. Ye required nahi lekin recommended hai.

Paragraphs

```
<p>This is my paragraph</p>
<p>This is another paragraph</p>
```

Paragraphs khud-ba-khud apas mein spacing bana lete hain.

Text Formatting Tags

Tag	Kaam
<code>text</code>	Bold text
<code><i>text</i></code>	Italic text
<code><big>text</big></code>	Text ko bara karta hai
<code><small>text</small></code>	Text ko chota karta hai
<code><sub>text</sub></code>	Subscript — jaise H ₂ O mein "2"
<code><sup>text</sup></code>	Superscript — jaise 10 ² mein "2"
<code>
</code>	Line break (single tag)
<code><hr/></code>	Horizontal line — content separate karne ke liye (single tag)

Tags ko nest bhi kar sakte hain: `<b><i>text</i></b>` — bold + italic dono.

Yaad rahe: HTML whitespace ko ignore karta hai. Chahe aap kitni bhi blank lines ya spaces file mein rakh dein, browser sirf tags ke andar wale content ko dekhta hai. Layout hamesha tags ke *order* se decide hota hai — jo pehle likha wo pehle show hoga.

5 Comments

```
<!-- Ye ek comment hai, browser ise ignore karega -->
```

Comments `<!--` se shuru hote hain aur `-->` par khatam. Ye multiple lines tak bhi phaila sakte hain. Browser inhe render nahi karta — ye sirf developers ke liye hote hain.

Do Use Cases

- **Notes chorna** — khud ke liye ya team ke liye reminders ("TODO: is ko fix karna hai")
- **Code comment-out karna** — kisi block ko temporarily disable karna bina delete kiye

Comments ko sparingly use karein. Best practice ye hai ke code khud itna clean ho ke usay explain karne ki zaroorat na pare — "let the HTML speak for itself".

6 Styles aur Colors

HTML elements ko style karne ke liye `style` attribute use hota hai. (Note: pura styling system CSS — Cascading Style Sheets — kehlata hai; ye sirf ek basic introduction hai.)

```
<p style="color: red; background-color: blue;">Text</p>
<body style="background-color: lightblue;">
```

Property	Kaam
<code>color</code>	Text ka color
<code>background-color</code>	Element ke peeche ka color

Agar kisi child element (jaise paragraph) par apna background-color diya ho, to wo parent (body) ki styling ko **override** kar dega. Colors ke naam ya hex codes (`w3schools` par color list) dono use ho sakte hain.

7 Layout Tags (Structure)

Har website generally 3 sections mein divide hoti hai: **Header** (top — navigation, logo), **Main** (asal content), aur **Footer** (bottom — links, social media).

```
<body>
  <header>
    <nav>...</nav>
  </header>
  <main>
    <article>
      <section>...</section>
      <section>...</section>
    </article>
    <aside>...</aside>
  </main>
  <footer>...</footer>
</body>
```

Tag	Kaam
<header>	Website ke top ka hissa — har page pe present, navigation/branding
<nav>	Navigational links (menu) store karne ke liye
<main>	Website ka main content
<article>	Ek independent block content — jaise blog post
<section>	Article ya page ko chote sections mein baantne ke liye
<aside>	Content jo main content se direct related nahi — jaise advertisement
<footer>	Website ka bottom hissa — links, copyright, social media

Ye Tags Kyun Zaroori Hain?

1. **Readability / Organization** — Code khud aapko ya doosre developers ko samajh aata hai (header kahan hai, main kahan hai)
2. **SEO (Search Engine Optimization)** — Google jaise search engines in specific tags ko use karke website ka structure asani se samajh sakte hain aur behtar rank de sakte hain

Generic `<div>` tags se layout banana possible hai, lekin specific semantic tags (header/main/footer/etc) use karna hamesha behtar practice hai.

8 Links (Anchor Tag)

Link banane ke liye `<a>` tag aur `href` attribute use hota hai.

External Link

```
<a href="https://www.google.com">Google's homepage</a>
```

Href mein hamesha `http://` ya `https://` zaroor include karein.

New Tab Mein Kholna

```
<a href="https://google.com" target="_blank">Google</a>
```

Internal / Relative Links

```
// Same folder mein file
<a href="page2.html">Page 2</a>

// Sub-folder ke andar file
<a href="dir1/page3.html">Page 3</a>

// File (image/pdf) ko link karna
<a href="cat.jpg">Cat Picture</a>
```

Links ke andar sirf text nahi, koi bhi HTML element daal sakte hain — jaise heading ya image:

```
<a href="https://google.com">
  <h1>Google's Homepage</h1>
</a>
```

Absolute URL (poora address jaise `https://google.com`) doosri websites ke liye, relative URL (sirf file ka naam/path) apni hi site ke doosre pages ke liye use hota hai.

9 Images

```

```

Attribute	Kaam
<code>src</code>	Image ki location — internet ka URL ya local file path
<code>alt</code>	Backup text jo dikhta hai agar image load na ho — hamesha add karein
<code>width</code> / <code>height</code>	Size control (pixels mein)

Aspect Ratio: Width aur height dono set karte waqt original aspect ratio maintain karna zaroori hai, warna image squeeze/stretch ho jayegi. Agar original 300×200 hai (3:2), to resize karte waqt bhi wahi ratio rakhein (jaise 100×66).

Image ko Clickable Link Banana

```
<a href="dinosaur.jpg">  
    
</a>
```

`<img>` ek **single tag** hai — koi closing tag nahi chahiye.

Local Video

```
<video src="tutorial.mp4" controls width="300"
  poster="thumb.jpg" autoplay loop>
  Sorry, your browser doesn't support videos.
</video>
```

Attribute	Kaam
<code>src</code>	Video file ka path
<code>controls</code>	Play/pause/volume controls dikhata hai (is ke bina video click se play nahi hoti)
<code>width</code> / <code>height</code>	Sirf width dena kaafi hai — height khud aspect ratio maintain kar leti hai
<code>poster</code>	Thumbnail image jo video play hone se pehle dikhti hai
<code>autoplay</code>	Page load hote hi video khud shuru ho jaye
<code>loop</code>	Video khatam hone par dobara se shuru ho jaye

Tags ke darmiyan likha text tab dikhta hai jab browser video format support na kare (rare case).

YouTube Video Embed Karna

YouTube video ke "Share" → "Embed" option se ek `<iframe>` code milta hai jo copy-paste karke apni site mein daal sakte hain:

```
<iframe width="560" height="315"
  src="https://www.youtube.com/embed/VIDEO_ID">
</iframe>
```

Iframe se video ke sath YouTube ke apne player controls (fullscreen, volume) bhi mil jate hain, aur width/height easily adjust ho sakti hai.

Unordered List (bina number/order ke)

```
<ul>
  <li>Apples</li>
  <li>Oranges</li>
  <li>Bananas</li>
</ul>
```

Har item ke aage default bullet/circle marker aata hai.

Ordered List (numbered)

```
<ol type="1">
  <li>Step One</li>
  <li>Step Two</li>
</ol>
```

type	value	Style
1		1, 2, 3 (default)
A		A, B, C (uppercase letters)
a		a, b, c (lowercase letters)
I		I, II, III (uppercase Roman)
i		i, ii, iii (lowercase Roman)

Lists ko nest bhi kar sakte hain — kisi `<li>` ke andar ek aur pura `<ul>` / `<ol>` daal sakte hain.

Description List (kam common, lekin useful)

```
<dl>
  <dt>Apples</dt>
  <dd>They are red</dd>
  <dt>Oranges</dt>
  <dd>They are orange</dd>
</dl>
```

`<dt>` = description term (item), `<dd>` = description detail (uski explanation).

90% waqt aap sirf `<ul>` aur `<ol>` hi use karenge — description list rarely use hoti hai.

```
<table>
  <caption>List of Numbers</caption>
  <thead>
    <tr>
      <th>Num1</th><th>Num2</th><th>Num3</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <td colspan="2">1</td><td>3</td>
    </tr>
  </tbody>
</table>
```

Tag	Kaam
<table>	Poori table ka container
<tr>	Table Row — ek horizontal entry
<td>	Table Data — ek column ka actual data
<th>	Table Header — column ka title (bold hota hai)
<caption>	Table ka overall title
<thead>	Header section ko group karta hai (organization ke liye)
<tbody>	Body/content section ko group karta hai

Colspan

`colspan="2"` attribute kisi ek `<td>` ko 2 columns ki jagah lene deta hai — baaki entries side mein shift ho jati hain.

Tables responsive hoti hain — window resize karne par text apne aap wrap ho jata hai. Rows/columns jitni chahein utni bana sakte hain.

13 Containers — Div aur Span

Container ek set of tags hota hai jo doosre HTML elements ko wrap karta hai. Bade projects mein CSS styling apply karne ke liye poore group ko wrap karna common practice hai.

Block vs Inline — Ye Concept Zaroor Samjhein

Block Elements

Poori width leta hai, agla element neeche chala jata hai. Example: `<p>` , `<div>` , `<h1>` - `<h6>`

Inline Elements

Sirf utni space leta hai jitni zaroorat ho, doosre inline elements ke sath sath line mein reh sakta hai. Example: `<a>` , `<span>` , `<b>`

Div — Block Container

```
<div>Div 1</div>
<div>Div 2</div> // har div apni nayi line par
```

Span — Inline Container

```
<span>Span 1</span>
<span>Span 2</span> // ek hi line mein saath rehte hain
```

`<div>` generally overall containers ke liye use hota hai (jaise pura section wrap karna), jabke `<span>` chote inline pieces of text ko style karne ke liye use hota hai (jaise paragraph ke andar ek word).

14 Input Tags aur Forms

Sirf HTML se input elements banane se wo **functional** nahi hote — user se actual data lene aur process karne ke liye **JavaScript** chahiye. HTML sirf UI elements define karta hai.

Text aur Password Input

```
<input type="text" value="Enter your username">
<input type="password"> // text ki jagah dots dikhte hain
```

Text Area (multi-line)

```
<textarea rows="10" cols="30">Enter a paragraph</textarea>
```

Common Input Types

Type	Kaam
<code>text</code>	Simple text box
<code>password</code>	Hidden text (dots)
<code>date</code>	Calendar/date picker
<code>email</code>	Email input
<code>range</code>	Slider
<code>file</code>	File upload chooser
<code>checkbox</code>	Multiple options select ho sakte hain
<code>radio</code>	Same <code>name</code> ho to sirf ek hi option select ho sakta hai
<code>submit</code>	Form submit karne ka button

Checkbox vs Radio Button

```
// Radio buttons – same 'name' de kar ek hi selectable banate hain
<input type="radio" name="button"> Option A
<input type="radio" name="button"> Option B
```

Form Tag

`<form>` tag inputs ke liye ek wrapper hota hai jo saara input data ikatha karke server ko bhej sakta hai (backend language ke saath).

Poori list ke liye [w3schools.com/tags/att_input_type](https://www.w3schools.com/tags/att_input_type) dekhein — bahut saare input types available hain.

15 iFrames

`<iframe>` ek dusri poori website ko apni website ke andar embed karne deta hai.

```
<iframe src="https://example.com" width="1000"
height="800" frameborder="0">
  Your browser doesn't support iframes.
</iframe>
```

Security Concern: Kuch bari websites (jaise Amazon) apni site ko iframe mein use hone se block kar deti hain, kyunke koi malicious website unki site ko iframe mein daal kar users ko trick kar sakti hai (phishing-type attack).

16 Meta Tags (SEO)

Meta tags `<head>` mein hote hain aur page ke baare mein metadata define karte hain.

```
<meta name="description" content="Your description">
<meta name="author" content="Your Name">
<meta name="keywords" content="html, tutorial, web dev">
<meta name="viewport"
content="width=device-width, initial-scale=1.0">
```

Meta Tag	Kaam
<code>description</code>	Search engines ke liye website ka short description (~160 characters ideal — zyada lamba ho to search engines ignore/penalize kar sakte hain)
<code>author</code>	Website ka author kaun hai
<code>keywords</code>	Comma-separated keywords (aajkal zyada search engines is ko ignore karte hain, lekin add karna theek hai)
<code>viewport</code>	Bahut important — mobile devices par website sahi tarah display hone ke liye zaroori hai, warna site zoomed-out/desktop version mein hi dikhegi

Ye 4-5 meta tags (charset, description, author, keywords, viewport) sabse zyada common aur important hain jo har website mein honi chahiye.

PART 2 · EXTENDED TOPICS

Web, Internet & Browser

Wo topics jo Part 1 mein miss hue the — HTML/CSS/JS roles, forms ke advanced tags, accessibility, aur phir web, internet aur browser kaise kaam karte hain — sab kuch neeche.

CONTENTS

17. HTML vs CSS vs JS — Roles Compared
18. Label, Select aur Button Tags
19. Native Form Validation Attributes
20. Tables — Kab Use Karni Chahiye (aur Kab Nahi)
21. Accessibility Basics
22. How the Web Works — Browser, DNS, HTTP
23. How the Web Works — HTTPS, JSON, WebSockets
24. How the Internet Works — LAN, Switch, Router
25. How the Internet Works — Packets aur Internet ki Structure
26. How the Internet Works — WAN aur VPN
27. How the Internet Works — ISP Hierarchy aur Peering
28. How the Browser Works — Rendering Engine

17 HTML vs CSS vs JS — Roles Compared

Ye teeno mil kar har modern webpage banate hain, lekin har ek ka kaam bilkul alag hai. In teeno ko confuse na karein — ye web development ka sabse basic aur zaroori concept hai.

HTML — Structure

HyperText Markup Language. Page ka *structure/skeleton* banata hai — kya heading hai, kya paragraph hai, kya button hai, kya input hai. Styling ke baare mein kuch nahi batata.

CSS — Presentation

Cascading Style Sheets. Page ka *look aur feel* control karta hai — colors, fonts, spacing, layout, animations. HTML elements ko "khubsurat" banata hai.

JavaScript — Behavior / Logic

Page mein *dynamic behavior aur interactivity* add karta hai — dropdown khulna, tabs switch hona, form validation, data fetch karna, click events handle karna. JS hi page ko "zinda" banata hai — page load hone ke baad jo bhi change hota hai, wo generally JS ki wajah se hota hai.

Sawal	HTML	CSS	JS
Ye kya hai?	Content ka structure	Content ki styling	Content ka behavior/logic
Example	<code><button></code>	<code>button { color: red; }</code>	<code>button.onclick = ...</code>
Browser ise kaise samajhta hai?	DOM (Document Object Model) banata hai	CSSOM (CSS Object Model) banata hai	V8/JS engine execute karta hai

Simple yaad rakhne ka tareeqa: **HTML = kya hai, CSS = kaisa dikhta hai, JS = kya karta hai.** Teeno independent languages hain, lekin browser inhein combine karke final page render karta hai.

18 Label, Select aur Button Tags

Label Tag — Input ke sath Association

`<label>` tag kisi input ko ek readable naam deta hai aur usay us input se "link" karta hai. Ye accessibility ke liye bahut important hai — screen readers is association ko use karte hain.

```
<label for="email">Email Address</label>
<input type="email" id="email">
```

Label ka `for` attribute aur input ki `id` match honi chahiye. Isse fayda ye hota hai ke user label pe click kare to bhi wo input focus/select ho jata hai — mobile pe checkboxes/radios ke liye khaas kaam ka hai.

Select — Dropdown Menu

```
<label for="country">Country</label>
<select id="country" name="country">
  <option value="pk">Pakistan</option>
  <option value="us">USA</option>
  <option value="uk">UK</option>
</select>
```

`<select>` ek dropdown banata hai jahan user multiple `<option>` mein se ek (ya `multiple` attribute lagane par kai) choose kar sakta hai. Jab bahut sari choices honi ho aur screen space bachana ho, tab radio buttons ki jagah select behtar hota hai.

Button Tag

Input ke `type="submit"` ke ilawa, HTML mein ek alag `<button>` tag bhi hota hai jo zyada flexible hai — is ke andar text ke sath icons, images, ya doosre HTML elements bhi daal sakte hain.

```
<button type="submit">Submit Form</button>
<button type="button">Just a Click (no submit)</button>
<button type="reset">Reset Form</button>
```

type value	Kaam
submit	Form submit karta hai (default agar form ke andar ho)
button	Kuch bhi submit nahi karta — sirf JavaScript se attach karne ke liye
reset	Form ke saare fields ko default values pe wapis le jata hai

19 Native Form Validation Attributes

HTML apne native attributes se hi basic validation de deta hai — bina JavaScript likhe. Browser khud user ko error message dikha deta hai agar input invalid ho.

```
<input type="text" required minlength="3" maxlength="20">
<input type="number" min="1" max="100">
<input type="email" required>
<input type="text" pattern="[A-Za-z]+" title="Sirf letters allowed">
```

Attribute	Kaam
<code>required</code>	Field khali nahi chor sakte — submit se pehle bharna zaroori
<code>minlength</code> / <code>maxlength</code>	Text ki minimum/maximum length control karta hai
<code>min</code> / <code>max</code>	Number ya date input ki minimum/maximum value
<code>pattern</code>	Ek regular expression deta hai jise input ki value match karni chahiye
<code>step</code>	Number/range input mein values kis interval mein badhein (jaise 0.5, 1, 5)
<code>title</code>	Validation fail hone par tooltip mein dikhne wala helper message

Ye sab **client-side validation** hai — user ka experience acha banata hai (turant feedback), lekin security ke liye kaafi nahi. Server par bhi hamesha dobara validate karna zaroori hai, kyunke client-side checks JavaScript se bypass ki ja sakti hain.

20 Tables — Kab Use Karni Chahiye

✓ Table Use Karein

- Tabular / grid data jismein rows aur columns ka real relationship ho
- Jaise: price comparison, schedule, spreadsheet-type data, results table

✗ Table Use Na Karein

- Sirf page ka **layout** banane ke liye (jaise header/sidebar/footer arrange karna)
- Purane zamane mein log layout ke liye tables use karte the — ye *bad practice* hai

Layout ke liye tables use karna accessibility aur SEO dono ke liye bura hai — screen readers is data ko "table" samajh kar confusing tareeqe se parhte hain, jabke wo actually sirf layout tha. Layout ke liye `<div>`, `<section>` aur CSS (Flexbox/Grid) use karein.

21 Accessibility Basics

Accessibility (a11y) ka matlab hai website ko un logon ke liye bhi usable banana jo screen readers use karte hain, keyboard-only navigate karte hain, ya kam dikhta hai.

Practice	Kyun Zaroori
<code>alt</code> text har image par	Screen reader image ki jagah ye text bolta hai — bina is ke visually-impaired user ko pata hi nahi chalta image mein kya hai
<code><label for="id"></code> association	Screen reader input ka naam bol sake, aur click se input focus ho sake
Heading order (<code>h1</code> → <code>h2</code> → <code>h3</code> ...)	Screen reader users headings se page navigate karte hain (jaise table of contents) — order skip hona confusing hota hai
Semantic landmarks	<code><header></code> , <code><nav></code> , <code><main></code> , <code><aside></code> , <code><footer></code> — screen readers in tags ko "landmarks" ki tarah use karte hain, taake user seedha "main content" ya "navigation" pe jump kar sake bina pura page sunay

Landmarks kya hote hain? Jo semantic layout tags aapne Section 7 mein seekhe the (header/nav/main/aside/footer), wahi tags accessibility tools ke liye "landmark regions" ban jate hain. Screen reader user ek shortcut key se "next landmark" pe jump kar sakta hai — is liye generic `<div>` ki bajaye semantic tags use karna sirf SEO ke liye nahi, blind/low-vision users ke liye bhi bohot zaroori hai.

Quick accessibility checklist: har image pe `alt`, har input pe associated `label`, headings hierarchy mein, aur page ke major sections semantic tags mein wrap hon.

22 How the Web Works — Browser, DNS, HTTP

Jab aap browser mein `academind.com` type karte hain, background mein bohot kuch hota hai us se pehle ke website screen par aaye.

Step 1: Domain → IP Address (DNS)

Har server ka asal address ek **IP address** hota hai (jaise `192.158.1.38`), jo insano ke type karne ke liye mushkil hai. Is liye hum human-readable **domain names** (jaise `academind.com`) use karte hain. Browser sabse pehle ek **DNS (Domain Name System)** server se contact karta hai — ye ek huge directory hai jo domain name ko us ke IP address mein translate karta hai.

DNS ko ek "phone book" ki tarah samajhein: aap naam (domain) jaante hain, DNS aapko number (IP address) deta hai.

Step 2: Request Bhejna

IP address milne ke baad, browser us server ko ek **request** bhejta hai. Request ek data package hota hai jismein:

- **URL** — kaunsa page chahiye
- **Request type/method** — GET (data lena), POST (data bhejna, jaise form submit), etc.
- **Headers** — extra metadata (browser type, accepted formats, etc.)
- **Body** — optional data (jaise form ka data)

Browser DevTools kholein (F12) → Network tab → koi bhi request select karein, aap ye sara data khud dekh sakte hain.

Step 3: Server ka Response

Server request ko process karta hai aur ek **response** wapis bhejta hai — jismein HTML code hota hai (jo page render karega) aur kuch metadata bhi. Ye HTML ya to server pe already stored hoti hai, ya server dynamically generate karta hai (jaise online shop mein cart ka content har user ke liye alag hoga).

HTTP aur HTTPS

Ye poora request/response ka format **HTTP** (HyperText Transfer Protocol) naam ke standard se define hota hai. **HTTPS** isi ka encrypted version hai — data end-to-end encrypted hota hai, matlab agar koi connection ko "sniff" bhi kare, use sirf unreadable data milega.

Aaj kal har modern website HTTPS use karti hai. Agar koi site sirf HTTP par ho (bina S ke), us par sensitive data (password, card number) na dalein.

Server-Side Languages aur Frameworks

Jab server ko HTML dynamically generate karni ho, us ke liye backend languages use hoti hain — jaise **Node.js**, **PHP**, **Python**, etc. Inke sath **frameworks** use hote hain jo common kaam (jaise incoming request parse karna) already kar dete hain, taake developer sirf apni business logic likhe.

Frontend mein bhi frameworks hote hain (React, Vue, Angular) jo JavaScript se page ko dynamically update karna asaan banate hain — bina har click par naya page load kiye.

Web = Sirf Websites Nahi

"Web" ka matlab sirf HTML pages nahi — mobile apps (Instagram, Twitter, Facebook) bhi isi tarah kaam karte hain: app ek request bhejta hai server ko, lekin HTML ki jagah wo **JSON** data mangta hai.

```
// JSON response ka example
{
  "username": "ali_dev",
  "posts": 42,
  "followers": 1200
}
```

App ka apna source code is JSON data ko parse karke screen par cards/lists ki tarah dikhata hai — request/response pattern same hai, sirf data format alag hai (HTML vs JSON).

WebSockets — Real-Time Communication

Normal request/response mein client hamesha pehle request bhejta hai. Lekin real-time apps (jaise chat, live notifications) mein server ko khud client ko naya data **push** karna hota hai — is ke liye **WebSockets** technology use hoti hai, jo ek persistent (hamesha khuli) connection banati hai.

Technology	Kaam
HTML	Page ka structure
CSS	Page ki styling
JavaScript	Page ka dynamic behavior
HTTP/HTTPS	Request/response ka standard format (encrypted ya na-encrypted)
DNS	Domain name ko IP address mein convert karta hai
JSON	Data exchange ka lightweight format (mobile apps ke liye common)
WebSockets	Real-time, do-tarfa (server → client push) communication

24 How the Internet Works — LAN, Switch, Router

"Web" chalta kis cheez par hai? Answer hai: **Internet** — jo actual physical cables, switches aur routers ka network hai. Ye samajhna zaroori hai ke data physically kaise safar karta hai.

Switch — Same Office/Home ke Devices Connect Karna

Agar aapke paas ek office mein 7 computers hon aur unhe aapas mein baat karni ho, to **switch** device use hoti hai. Computers ko switch se **copper cable** (Cat5/Cat6) ya **fiber optic cable** se connect kiya jata hai (fiber zyada fast hota hai). Wireless version ko **Access Point** kehte hain.

Switch sirf cable/wireless se connect hone wale devices ke darmiyan ek **LAN (Local Area Network)** banata hai — ek restricted physical area (ghar, office) tak mehdood.

Packet — Data ka Basic Unit

Jab ek computer doosre ko message bhejta hai, us data ko **packet** kehte hain. Switch packet ke andar dekh kar uski destination samajh leta hai aur sahi computer tak forward kar deta hai.

Router — Internet Se Connect Karna

Switch sirf LAN ke andar communication allow karta hai — internet se connect karne ke liye **router** chahiye. Router LAN ko ISP (Internet Service Provider) ke cable se connect karta hai, jo aage internet se juda hota hai.

Home router ek common device hai jo switch + router dono ka combination hota hai — chote environments (ghar, choti office) ke liye kaafi hai.

Device	Kaam
Switch	Same LAN ke andar devices ko connect karta hai (cable se)
Access Point	Same LAN ke andar devices ko wirelessly connect karta hai
Router	Alag-alag networks (LANs) ko aapas mein aur internet se connect karta hai

Internet = Network of Networks

Duniya bhar mein lakhon LANs hain (har ghar, har office ka apna LAN). Jab hum inhein routers ke zariye aapas mein connect karte hain, wahi structure **internet** kehlata hai. Bookish definition: "**Internet is the network of networks.**"

Kai Routers Kyun Chahiye? (Ek Bara Router Kyun Nahi)

- **Single point of failure** — agar sirf ek hi router ho aur wo kharab ho jaye, to pura internet down ho jayega
- **Cable length problem** — duniya ke doosre kinare tak ek hi router se cable lena impractical hai
- **Load balancing** — distributed routers traffic ko efficiently baant sakte hain

Is liye internet ek **distributed structure** hai — hazaron routers jo duniya bhar mein failay hue hain. Continents ke darmiyan ye connection **undersea fiber-optic cables** (samandar ke neeche cables) se hota hai — 99% international internet traffic inhi cables se guzarta hai.

Routing Table aur Forwarding

Har router ke paas ek **routing table** hoti hai jo batati hai ke packet ko kaunse port se aage bhejna hai. Router hamesha wo path choose karta hai jo *fastest* ho — hamesha "sabse kam stops" wala path nahi, balke jahan traffic congestion kam ho (isay **congestion control** kehte hain).

Packet apni manzil tak pohanchne ke liye kai routers se guzarta hai, aur har router independently decide karta hai ke agla step kya hoga — is process ko **forwarding** kehte hain.

Streaming aur Distributed Servers

Jab hum koi video (jaise YouTube/Udemy) dekhte hain, poori video ek sath download nahi hoti — **piece by piece** bheji jati hai, isay **streaming** kehte hain. Bari companies (Google, Amazon) apne **servers** duniya bhar mein distribute karte hain, taake user ko sabse qareeb server se jawab mile — is se speed behtar hoti hai aur single point of failure bhi avoid hota hai.

WAN — Wide Area Network

WAN mukhtalif LANs ka combination hota hai — jaise ek company ke do offices, jo duniya ke alag hisson mein hon, unhein ek network ki tarah connect karna. Chote scale par, agar do offices sirf 100 meters door hon aur unke switches direct connect kiye jayein, use **CAN (Campus Area Network)** kehte hain (universities mein common).

Public Internet vs Private Network

Internet ek **public network** hai — koi bhi is se connect ho sakta hai, is liye hackers ka khatra hamesha rehta hai. Agar company ka sensitive data direct internet par bheja jaye, to koi bhi is ko intercept kar sakta hai.

VPN aur Tunneling

VPN (Virtual Private Network) is problem ka common solution hai. VPN ka sabse important feature **tunneling** hai — jo public internet ke upar ek "private tunnel" jaisa secure connection banata hai.

Tunneling ka concept: Original packet ko encrypt karke ek doosre packet ke andar "pack" kar diya jata hai (jaise letter ko envelope mein daalna). Beech mein koi is packet ko dekh le tab bhi sirf unreadable/encrypted data nazar aayega.

Do offices ke routers ke darmiyan ye tunneling setup karna **site-to-site VPN** kehata hai. Isay **public WAN** bhi kaha jata hai (kyunke ye public internet infrastructure use karta hai, bas encrypted tareeqe se).

Private WAN

Agar company ISP se ek dedicated line kiraye par le (jo sirf unki apni company use kare), use **private WAN** kehte hain. Ye zyada secure hota hai (kyunke packet kabhi public internet se nahi guzarta), lekin bohot costly hota hai — is liye zyada tar companies public WAN (VPN) hi use karti hain.

Network Type	Security	Cost
LAN (same cable/office)	Sabse secure — kabhi internet par nahi jata	Kam
Private WAN (dedicated ISP line)	Bohot secure	Zyada
Public WAN (VPN over internet)	Acha (tunneling + encryption ki wajah se)	Kam (sabse common choice)

Yaad rahe: **LAN banane ke liye switch** use hota hai, jabke **WAN banane ke liye router** — kyunke WAN ka matlab hi alag-alag networks ko connect karna hai.

27 How the Internet Works — ISP Hierarchy aur Peering

ISP (Internet Service Provider) wo company hai jo humein internet se connect karne ki service deti hai (paise le kar). Poori duniya mein hazaron ISPs hain, aur ye ek hierarchy mein organized hain.

ISP Level	Scope
Local ISP	Ek neighborhood/mohalla ke andar communication (sabse pehla step internet se connect hone ka)
Regional ISP	Ek country ke andar mukhtalif cities ko connect karta hai
Global ISP	Mukhtalif countries/continents ko connect karta hai (undersea cables ke zariye)

Local ISP ka office **PoP (Point of Presence)** kehlata hai — yahan routers, switches aur servers hote hain. Aapke ghar se jo cable ISP deta hai, wo isi PoP tak jati hai.

Global ISPs ka aapas mein connected structure **Internet Backbone** kehlata hai, aur jahan multiple global ISPs efficiently connect hote hain use **IXP (Internet Exchange Point)** kehte hain.

Peering — Bari Companies ka Shortcut

Google jaisi bari companies apna traffic fast karne ke liye **peering** use karti hain — ye Local ISP ke sath direct connection banati hain, taake unka data poori ISP hierarchy (regional/global) se guzarne ki bajaye seedha pohanche.

Yahi wajah hai ke YouTube (Google ki property) zyada smooth stream hoti hai — Google ke pass peering aur distributed servers dono hain. Chote platforms jinke pass ye infrastructure nahi, unme kabhi kabhi buffering dekhne ko milti hai.

Ek zaroori concept: aap zaroori nahi ke sirf Local ISP se hi connect hon — agar zaroorat aur budget ho, to Regional ya Global ISP se bhi seedha connection le sakte hain, lekin normal users ke liye Local ISP hi kaafi hoti hai.

28 How the Browser Works — Rendering Engine

Browser sirf ek simple app nahi — ye khud ek chota operating-system jaisa hai: is ke paas apna data storage (cookies, local storage), networking capability, timers, aur do bare engines hote hain — **JS Engine** aur **Rendering Engine**. Yahan hum rendering engine samjhenge, jo HTML/CSS ko screen par convert karta hai.

Step-by-Step: HTML se Screen Tak

1. **Raw Bytes** — file disk/network se sirf 0s aur 1s ki form mein aati hai
2. **Character Encoding** — bytes ko characters mein convert kiya jata hai (jaise UTF-8 ke hisaab se)
3. **Tokenization** — characters se meaningful "words" nikale jate hain (jaise `<h1>`, `<p>`, `<body>`)
4. **Object Creation** — har token ek object banta hai (tag ka naam, attributes, value)
5. **DOM Construction** — in objects ko parent/child/sibling relationships ke sath ek tree structure diya jata hai — yehi **DOM (Document Object Model)** hai

Jab aap JavaScript se `document.getElementsByTagName('h2')` jaisa code likhte hain, jo आपको wapis milta hai wo isi DOM (node list) se aata hai — direct HTML text se nahi.

CSS ka Bhi Wahi Process — CSSOM

CSS bhi isi tarah raw bytes → characters → tokens → objects → tree structure ke marahil se guzarti hai, jis se **CSSOM (CSS Object Model)** banta hai. DOM aur CSSOM dono independent hote hain aur parallel bante hain.

Render Tree aur Painting

DOM aur CSSOM ko combine karke ek **Render Tree** banaya jata hai. Is stage par browser mathematical calculations karta hai ke har element screen par kitni jagah lega (layout). Is ke baad final step aata hai: **Painting** — jahan actual pixels screen par draw hote hain.

JavaScript Rendering Ko "Block" Kar Sakti Hai

Jab browser ko `<script>` tag milta hai, wo DOM/CSSOM banane ka kaam **ruk deta hai** aur pehle JavaScript ko process karta hai — kyunke JS DOM/CSS ko modify kar sakti hai. Aur agar CSSOM abhi complete nahi hui, to JavaScript ka execution bhi **ruk jata hai** jab tak CSSOM ready na ho.

Is wajah se modern practice ye hai ke JavaScript ko `async` ya `defer` attributes ke sath page ke end mein load kiya jaye, taake HTML/CSS pehle render ho jayein aur user ko jaldi content dikhe — JS baad mein interactivity add kar deti hai.

Yaad rakhne wali cheez: **Rendering Engine** zyada tar C++ mein likhi hoti hai (jaise Chrome ka Blink, Safari ka WebKit) — HTML/CSS khud "programming languages" nahi hain, balke ye markup hai jise ek powerful C++ engine parse karke pixels mein convert karta hai.

PART 3 • PROTOCOLS DEEP DIVE

Internet Protocols & DNS

Internet ke core protocols ki deep dive — TCP/IP, ports/sockets, SSL/TLS encryption, HTTP ki पूरी history (0.9 se HTTP/2 tak), aur DNS resolution ka mukammal step-by-step process.

CONTENTS

- 29. Internet Protocols — TCP/IP, Ports aur Sockets
- 30. SSL/TLS — Encryption, Certificates aur Handshake
- 31. HTTP Ki History — 0.9 se HTTP/1.1 Tak
- 32. HTTP Ki History — SPDY aur HTTP/2
- 33. DNS Deep Dive — Nameservers aur Resolution Process
- 34. DNS Deep Dive — Query Types aur Caching

29 Internet Protocols — TCP/IP, Ports aur Sockets

Protocol rules aur standards ka ek set hota hai jo define karta hai ke devices aapas mein data kaise exchange karte hain. Internet is liye kaam karta hai kyunke har device same standardized protocols follow karta hai — chahe wo kisi bhi company ne banaya ho.

IP aur TCP — Do Core Protocols

Protocol	Kaam
IP (Internet Protocol)	Packet ko sahi destination tak <i>route</i> karta hai — jaise dak (post) ka address system
TCP (Transmission Control Protocol)	Packets <i>reliably</i> aur <i>correct order</i> mein pohanchte hain, ye ensure karta hai — agar koi packet kho jaye to dobara bhejta hai
UDP (User Datagram Protocol)	TCP se tez hai lekin reliability guarantee nahi karta — video calls, gaming jaise fast use-cases ke liye

In dono ko combine karke **TCP/IP** kaha jata hai — ye poore internet ki underlying communication ka foundation hai.

Ports — Application Identify Karna

Ek hi device par bohot saare applications internet use kar rahe ho sakte hain (browser, email client, games). **Port** number batata hai ke data kis specific application/service ke liye hai.

Port	Default Use
80	HTTP (bina encryption)
443	HTTPS (encrypted)

Sockets aur Connections

- **Socket** = IP address + Port number ka combination — ek specific communication endpoint
- **Connection** = jab do sockets aapas mein baat karne ke liye connect hote hain (jaise TCP ka 3-way handshake)
- **Data Transfer** = connection banne ke baad data *segments* mein bheja jata hai, har segment ke sath ek sequence number hota hai taake sahi order maintain rahe

Application-level protocols jaise **HTTP**, **FTP (file transfer)**, **SMTP (email)** sab TCP/IP ke upar hi bante hain — TCP/IP "neechे ki foundation" hai, HTTP jaise protocols "upar ki application layer" hain.

SSL (Secure Sockets Layer) aur uska modern successor **TLS (Transport Layer Security)** wo protocols hain jo internet par data ko encrypt karte hain — yehi HTTPS ke "S" ke peeche ki technology hai.

Teen Core Concepts

Concept	Tafseel
Certificate	Server ki identity prove karta hai. Ek trusted third-party (Certificate Authority) is certificate ko sign karti hai taake browser server par bharosa kar sake
Handshake	Client aur server milkar decide karte hain ke konsa encryption algorithm use hoga aur connection kaise secure banega — ye sab actual data bhejne se pehle hota hai
Encryption	Handshake complete hone ke baad, sara data agreed-upon algorithm se encrypt ho kar transmit hota hai

Jab aap kisi HTTPS website par jaate hain, browser address bar mein ek **padlock icon** dikhata hai — iska matlab hai connection SSL/TLS se secure hai aur beech mein koi bhi data ko "sniff" kare, to use sirf encrypted (unreadable) data milega.

Developer Ke Liye Zaroori Baatein

- Sensitive data (login, payment info) hamesha SSL/TLS ke sath transmit karein
- Apne server ke liye valid SSL/TLS certificate obtain aur maintain karein
- Best practices follow karein taake connection properly configured aur secure rahe

SSL/TLS bina, koi bhi network ke beech mein baithi party (jaise public WiFi par koi hacker) aapka data plain-text mein parh sakti hai — is liye modern web par HTTPS ab practically mandatory hai.

31 HTTP Ki History — 0.9 se HTTP/1.1 Tak

HTTP (HyperText Transfer Protocol) waqt ke sath evolve hua hai. Har version ne pichle version ki limitations solve ki.

HTTP/0.9 (1991) — "The One Liner"

Sab se simple version — sirf ek method (`GET`) tha. Koi headers nahi the, response hamesha sirf HTML hoti thi, aur request fulfill hote hi connection band ho jati thi.

```
GET /index.html
```

HTTP/1.0 (1996)

Bohot bara improvement: ab images, videos, plain text — har content type support hua. `POST` aur `HEAD` methods add hue, aur pehli baar **headers** aur **status codes** introduce hue.

```
GET / HTTP/1.0
Host: cs.fyi
User-Agent: Mozilla/5.0
```

Bara masla: Har request ke liye ek naya TCP connection kholna parta tha, aur fulfill hone ke baad connection band ho jati thi. Ek webpage jismein 20 images/files hon, to 20 alag connections khulti thin — ye bohot slow tha (har connection ke liye 3-way handshake ka overhead).

HTTP/1.1 (1999) — Bara Sudhar

Feature	Fayda
Persistent Connections	Connection default se khuli rehti hai — baar baar nayi connection nahi kholni parti
Pipelining	Client multiple requests ek sath bhej sakta hai bina pehle response ka wait kiye
Chunked Transfer	Server dynamic content ko piece-by-piece bhej sakta hai jab total size pehle se pata na ho
New Methods	<code>PUT</code> , <code>PATCH</code> , <code>DELETE</code> , <code>OPTIONS</code> add hue
Host Header Required	Same server par multiple websites host karna mumkin hua

HTTP/1.1 ka masla ye tha ke ek connection par ek waqt sirf ek hi request "outstanding" ho sakti thi — is se **head-of-line blocking** hoti thi (ek slow request baaki sab ko roke rakhti thi). Is wajah se developers ko workarounds (sprite-sheets, combined CSS/JS files) use karne parte the.

SPDY (2009) — Google ka Experiment

Google ne latency kam karne ke liye ek naya protocol banaya jise **SPDY** kaha gaya. Iska core idea tha: bandwidth badhane se performance thori der behtar hoti hai phir plateau ho jati hai, lekin **latency kam karne se performance hamesha behtar hoti rehti hai**. SPDY multiplexing, compression aur security features laya — jo baad mein HTTP/2 ki buniyad bana.

HTTP/2 (2015)

SPDY se inspired ho kar HTTP/2 launch hua, jo low-latency content transport ke liye design kiya gaya.

Feature	Tafseel
Binary Protocol	Text ki bajaye binary format use karta hai — parse karna asaan aur fast hai (lekin insano ke liye directly readable nahi)
Multiplexing	Ek hi TCP connection par multiple requests/responses <i>ek sath (asynchronously)</i> chal sakti hain — head-of-line blocking khatam
Header Compression (HPACK)	Repetitive headers (jaise User-Agent) baar baar poori nahi bheji jatin — dono taraf ek shared table maintain hoti hai
Server Push	Server pehle se andaza laga kar client ko wo resources bhej deta hai jo use "maang-ne se pehle" chahiye hongi (<code>PUSH_PROMISE</code> frame)
Request Prioritization	Client bata sakta hai ke kaunsi request zyada important hai, taake server resources us hisaab se allocate kare

Frames aur Streams

HTTP/2 mein har request/response ek unique **Stream ID** rakhta hai aur chote binary pieces mein tut jata hai jinhein **Frames** kehte hain (jaise HEADERS frame, DATA frame). Client ke requests odd-numbered stream IDs use karte hain, server ke responses even-numbered.

Security: HTTP/2 ki spec mein encryption officially mandatory nahi, lekin practically har major browser sirf TLS ke sath hi HTTP/2 support karta hai — is liye HTTP/2 = HTTPS + Binary + Multiplexing, yehi real-world norm hai.

33 DNS Deep Dive — Nameservers aur Resolution Process

DNS ko internet ki "**phonebook**" kaha jata hai — ye human-readable domain names (jaise `example.com`) ko machine-friendly IP addresses (jaise `192.168.1.1`) mein translate karta hai.

4 Server Types Jo DNS Lookup Mein Involve Hote Hain

Server	Kaam (Library Analogy)
DNS Recursor	Ek "librarian" jo aapki taraf se poori book dhoondne ka zimma leta hai — client (browser) se query leta hai aur baaki servers se contact karta hai
Root Nameserver	Library ka "index" — batata hai ke kaunse rack (TLD server) par jana hai
TLD Nameserver	Ek specific "rack" (jaise <code>.com</code> ya <code>.net</code>) — domain ki last portion ko host karta hai
Authoritative Nameserver	Rack par rakhi "dictionary" — final answer deta hai: is domain ka asal IP address kya hai

DNS Lookup Ke 8 Steps (Jab Kuch Cached Na Ho)

1. User browser mein `example.com` type karta hai → query DNS **recursive resolver** tak pohanchti hai
2. Resolver ek **root nameserver** ko query karta hai
3. Root server jawab deta hai: **TLD server** (jaise `.com`) ka address
4. Resolver ab `.com` TLD server ko query karta hai
5. TLD server jawab deta hai: domain ke **authoritative nameserver** ka address
6. Resolver authoritative nameserver ko query karta hai
7. Authoritative nameserver asal **IP address** wapis bhejta hai
8. Resolver ye IP address browser ko de deta hai

Is ke baad browser us IP address par seedha **HTTP request** bhejta hai, aur server webpage wapis bhejta hai — tab jakar page render hota hai.

Ye poora process milliseconds mein hota hai, aur "behind the scenes" hota hai — user ko bilkul pata nahi chalta ke itne saare servers se guzar kar unki request pori hui hai.

Recursive Resolver vs Authoritative Server — Farq

Recursive DNS Resolver

Query ke **shuru** mein hota hai. Client ki taraf se poori "digging" karta hai — jab tak authoritative server na mil jaye tab tak multiple requests bhejta rehta hai.

Authoritative DNS Server

Query ke **end** mein hota hai. Actual DNS records ka "source of truth" hai — final IP address yahi se milta hai.

3 Types of DNS Queries

Query Type	Kaam
Recursive Query	Client DNS server se maang karta hai ke wo <i>khud</i> poora record dhoond kar de, ya error de
Iterative Query	Server apna best jawab deta hai; agar match na mile to agle (niche ke level ke) server ka reference de deta hai — client khud agla step karta hai
Non-Recursive Query	Server ke paas already jawab available hota hai (cache ya khud authoritative hone ki wajah se) — turant jawab milta hai

DNS Caching — Speed Ka Raaz

Har DNS record ke sath ek **TTL (Time To Live)** hota hai — ye batata hai record kitni der cache mein reh sakta hai. Caching kai levels par hoti hai:

1. **Browser Cache** — browser khud thodi der ke liye DNS records yaad rakhta hai (Chrome mein `chrome://net-internals/#dns` par dekh sakte hain)
2. **OS-Level Cache** — operating system ka "stub resolver" apna chota cache maintain karta hai
3. **ISP Resolver Cache** — aapke ISP ka recursive resolver bhi records cache karta hai taake baar baar root/TLD servers tak na jana pare

Jitni zyada caching hoti hai, utni kam "hops" lagti hain aur website utni tezi se load hoti hai. Yehi wajah hai ke pehli baar kisi naye domain ko visit karna thora slow lagta hai, lekin dobara visit karna fast hota hai — kyunke DNS record already cached hoti hai.